

The C++ OOP Handbook

A book-style guide to Object-Oriented Programming in C++ — not just *what* each feature does, but *how it actually works under the hood*: memory layout, the vtable, dynamic dispatch, and where the classic pitfalls (slicing, missing virtual destructors, shallow copies) actually come from. Every major concept follows the same shape: definition → why it exists → a real example → common mistakes → what interviewers listen for. Targets **C++17**, with C++11/14/20 notes called out explicitly wherever the standard changed something relevant.

12 chapters, foundations → advanced

Memory-layout & vtable diagrams

Large interview Q&A bank + practice problems

Companion single-file LaTeX edition

Updated Sep 2026

HOW THIS BOOK IS ORGANIZED

- **Chapters 1–2** build the vocabulary: classes, objects, the four pillars, as C++ actually spells them.
- **Chapter 3** is the one most books skip: what a virtual call, a vtable, and object slicing actually look like in memory.
- **Chapters 4–8** cover the machinery real C++ code leans on daily: RAII, smart pointers, inheritance trade-offs, operator overloading, const-correctness.
- **Chapters 9–12** are for using what you've learned: diagrams, a large interview bank, practice problems, and a cheat sheet.

HOW EVERY CONCEPT IS TAUGHT

- **Definition** — in plain language, the first time a term appears.
- **Why it exists** — the problem it was invented to solve.
- **A real, minimal example** — short enough to read in one glance.
- **Common mistakes** — the specific way beginners get it wrong.
- **Interview notes** — what an interviewer is actually listening for when they ask about it.

CONTENTS

01 Foundations	3	02 The Core OOP Pillars in C++	6
03 How It Works Internally	10	04 Special Member Functions & RAII	14
05 Advanced Inheritance	19	06 Operator Overloading	22
07 Const-Correctness & API Design	24	08 Static Members, Friends & Tools	26
09 Diagrams & Mental Models	28	10 Interview Question Bank	35
11 Practice Problems	46	12 Quick Revision & Glossary	50

01 Foundations

Everything else in this book is a refinement of the ideas in this chapter. Read it even if you think you already know it — the definitions here are used precisely, on purpose, throughout the rest of the book.

1.1 Class vs. object

A **class** is a blueprint — a type you define that describes what data (**state**) and what operations (**behavior**) something will have. An **object** is one concrete instance of that class, living somewhere in memory, with its own copy of the state.

```
class BankAccount {
public:
    explicit BankAccount(double balance) : balance_(balance) {}
    void deposit(double amount) { balance_ += amount; }
    double balance() const { return balance_; }
private:
    double balance_; // state
};

BankAccount alice(100.0); // alice is an OBJECT of class BankAccount
BankAccount bob(50.0);   // a completely separate object, own balance_
alice.deposit(25.0);      // alice.balance_ becomes 125.0; bob is unaffected
```

Why it exists: before classes, programmers manually kept a struct's data and the free functions that operated on it in sync by convention alone — nothing stopped unrelated code from reaching in and corrupting that data. A class binds data and the operations on it into one unit, so the type itself can enforce its own rules (Section 1.5).

COMMON MISTAKE

Confusing the class (one definition, compiled once) with its objects (as many as you create at runtime). "How much memory does a BankAccount use?" is a question about the class; "how many bank accounts exist right now?" is a question about objects.

INTERVIEW NOTES

Interviewers use this as a warm-up to confirm you don't conflate struct and class — in C++ they are *identical* except for one default: struct members default to public, class members default to private. Everything else (inheritance, methods, templates) works the same on both.

1.2 Access specifiers

SPECIFIER	VISIBLE TO
public	Anyone with an object of this type
protected	This class and any class that inherits from it (Chapter 2)
private	Only this class's own member functions (and its friends, Chapter 8)

In `BankAccount` above, `balance_` is `private` — no code outside the class can write `alice.balance_ = -1000000`; directly. All access is forced through `deposit()` and `balance()`, which is exactly the point (Section 1.5).

1.3 Constructors and destructors

A **constructor** is a special member function, named exactly like the class, that runs automatically when an object is created — its job is to establish the object's **invariants** (Section 1.5) before anyone can touch it. A **destructor** (`~ClassName()`) runs automatically when an object's lifetime ends — its job is cleanup (releasing memory, closing files, unlocking mutexes).

```
class FileHandle {
public:
    explicit FileHandle(const std::string& path) {
        file_ = std::fopen(path.c_str(), "r"); // constructor: acquire the resource
    }
    ~FileHandle() {
        if (file_) std::fclose(file_); // destructor: release it, always
    }
private:
    FILE* file_;
};
```

C++ gives you several kinds of constructor, each answering a different question about how an object comes into existence:

CONSTRUCTOR	ANSWERS	SIGNATURE SHAPE
Default	"Make one with no arguments"	<code>ClassName()</code>
Parameterized	"Make one from these values"	<code>ClassName(int x, ...)</code>
Copy	"Make one that's a copy of that other one"	<code>ClassName(const ClassName&)</code>
Move	"Make one by cannibalizing that temporary"	<code>ClassName(ClassName&&)</code>

Copy and move constructors are covered in full in Chapter 4 (Rule of 5) — they're important enough to deserve their own chapter, not a one-line mention here.

1.4 Member initializer lists

```
// Prefer this:
BankAccount(double balance) : balance_(balance) {}

// over this:
BankAccount(double balance) { balance_ = balance; }
```

The initializer-list form (after the `:`) directly *constructs* each member with the given value. The body-assignment form first **default-constructs** every member, then immediately overwrites it — for a primitive type like `double` the difference is negligible, but for a member that's itself a class (say, a `std::string` or another object with a non-trivial constructor), the body-assignment form does real, wasted work: build an empty one, then throw it away and assign a new value. Two situations make the initializer list *mandatory*, not just preferred: initializing a `const` member, and initializing a reference member — neither can be assigned to after construction, only initialized during it.

COMMON MISTAKE

Members are initialized in the order they're *declared in the class*, not the order they appear in the initializer list. Listing them out of declaration order doesn't reorder execution — it just makes the code confusing, and some compilers will warn about it.

1.5 Encapsulation and invariants

Encapsulation is bundling data with the code that operates on it, and restricting direct access to that data from outside (Section 1.2). The actual payoff is protecting an **invariant** — a condition that must always be true for an object to be in a valid state. For `BankAccount`, a reasonable invariant might be "balance never goes negative." If `balance_` were `public`, nothing could stop `alice.balance_ = -50;` from silently breaking that invariant anywhere in the program. Because it's `private` and only reachable through `deposit()`/a guarded `withdraw()`, the class itself is the single place that invariant can be enforced or violated — which makes bugs findable.

INTERVIEW NOTES

A weak answer defines encapsulation as "hiding data." A strong answer names the actual payoff: it lets a class *guarantee an invariant*, because there's exactly one place (the class's own member functions) that can ever change the data.

1.6 Abstraction via interfaces

Abstraction is exposing only what a caller needs to know, hiding how it's actually done. In C++, the primary tool for this is an **abstract class** — a class with at least one **pure virtual function** (a method declared with `= 0`, with no implementation in that class), which cannot be instantiated directly and exists purely to define an interface other classes must implement (covered fully in Chapter 2).

```
class Shape {    // an abstract class -- an interface
public:
    virtual double area() const = 0;    // pure virtual -- no body here
    virtual ~Shape() = default;
};
// Shape s;    -- ERROR: cannot instantiate an abstract class
```

Code written against `Shape*` never needs to know whether it's actually pointing at a `Circle` or a `Rectangle` — that's the abstraction: the caller depends on the interface's *intent* ("give me your area"), not any concrete implementation's details.

CHAPTER 1 RECAP

- Class = blueprint; object = one instance with its own state
- `struct` defaults public, `class` defaults private — otherwise identical
- Constructor establishes invariants; destructor cleans up, automatically
- Prefer member-initializer lists; mandatory for `const`/reference members
- Encapsulation's real payoff is protecting an invariant, not just "hiding data"
- Abstraction in C++ = an abstract class with pure virtual functions

02 The Core OOP Pillars in C++

Encapsulation and abstraction were Chapter 1. This chapter covers the other two pillars — inheritance and polymorphism — and the C++-specific vocabulary (`virtual`, `override`, `final`) that makes them work.

2.1 Inheritance and the three access levels

Inheritance lets one class (the **derived class**) acquire the members of another (the **base class**), modeling an "is-a" relationship. C++ has a feature most other OOP languages don't: you choose *how* the base class's access levels carry into the derived class.

```
class Animal { public: void breathe() {} };

class Dog : public Animal {}; // public members of Animal stay public in Dog
class Cat : protected Animal {}; // public members of Animal become protected in Cat
class Fish : private Animal {}; // public members of Animal become private in Fish
```

INHERITANCE TYPE	MODELS	HOW COMMON IN PRACTICE
public	Genuine "is-a" — a Dog really is usable anywhere an Animal is expected	The overwhelming default; almost always what you want
protected	"Is-implemented-in-terms-of," visible to further subclasses only	Rare in practice
private	"Is-implemented-in-terms-of," hidden from everyone outside	Rare — usually composition (Chapter 5) expresses this better

WHEN TO AVOID INHERITANCE

If you catch yourself inheriting just to reuse a few methods, with no real "is-a" relationship, that's a warning sign. A `Stack` is *not* a `Vector` just because it can reuse a vector's storage — it *has* one internally. Chapter 5 covers this decision in depth.

2.2 Polymorphism: compile-time vs. runtime

Polymorphism means "many forms" — the same interface behaving differently depending on context. C++ has two entirely different mechanisms for this, and conflating them is a common interview stumble:

	COMPILE-TIME (STATIC)	RUNTIME (DYNAMIC)
Mechanism	Function/operator overloading, templates	virtual functions
Resolved	By the compiler, at compile time	At runtime, via the vtable (Chapter 3)
Cost	Zero runtime overhead	One indirect call per virtual dispatch
Example	<code>print(int)</code> vs. <code>print(string)</code>	<code>shape->area()</code> calling the right override

When people say "polymorphism" in an OOP interview without qualification, they almost always mean **runtime polymorphism** — the rest of this chapter is about that.

2.3 Virtual functions, overriding, and overloading

A **virtual function** is a member function marked `virtual` in the base class, telling the compiler: "don't decide which version to call until runtime — look at the actual object's type." **Overriding** is a derived class providing its own implementation of a base class's virtual function, with an identical signature. This is completely different from **overloading** — multiple functions with the *same name* but *different parameter lists*, resolved at compile time (Section 2.2).

```
class Shape {
public:
    virtual double area() const { return 0.0; }    // virtual: overridable
    virtual ~Shape() = default;
};
class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; } // OVERRIDING
private:
    double radius_;
};

// OVERLOADING (compile-time, unrelated to virtual):
void print(int x);
void print(const std::string& x);
```

Calling through a base pointer/reference is what actually exercises dynamic dispatch:

```
Shape* s = new Circle(2.0);
s->area();    // calls Circle::area() at runtime -- NOT Shape::area() --
              // even though the compile-time type of s is Shape*
```

2.4 Pure virtual functions and abstract classes

Covered by example in Section 1.6. Formally: `virtual double area() const = 0;` is a **pure virtual function** — it has no implementation in that class (though, non-obviously, it's legal for it to have one that derived classes can explicitly call via `Shape::area()`). A class with at least one pure virtual function is **abstract**: it cannot be instantiated, and any concrete derived class *must* override every pure virtual function it inherits, or it too remains abstract.

2.5 Virtual destructors: why required, what breaks without one

```

class Base {
public:
    ~Base() { std::cout << "~Base\n"; }          // NOT virtual -- the bug
};
class Derived : public Base {
public:
    ~Derived() { delete[] buffer_; std::cout << "~Derived\n"; }
private:
    int* buffer_ = new int[100];
};

Base* b = new Derived();
delete b;
// Output: "~Base" -- ~Derived() NEVER RUNS. buffer_ leaks. Undefined behavior
// territory: only the static type (Base) is used to pick which destructor
// runs, because there's no vtable entry telling it otherwise.

```

Why this happens: without `virtual`, destructor selection is resolved statically — at compile time, from the pointer's declared type (`Base*`), exactly like a non-virtual function call. The fix is one word:

```

class Base {
public:
    virtual ~Base() { std::cout << "~Base\n"; }    // now it's in the vtable
};
// delete b; now correctly prints "~Derived" then "~Base"

```

THE RULE

If a class has *any* virtual function, or is ever going to be deleted through a base pointer, give it a virtual destructor. If a class is never going to be used polymorphically (no virtual functions at all, never inherited from with dynamic deletion in mind), a non-virtual destructor is fine and slightly cheaper — but this is the exception, not the default assumption.

INTERVIEW NOTES

This is one of the single most commonly asked C++ OOP questions, precisely because getting it wrong produces a real, silent resource leak rather than a compiler error — interviewers want to hear "undefined behavior / resource leak," and specifically *why* (static resolution of the destructor call), not just "it's bad practice."

2.6 override and final

`override` (C++11) is a keyword you add to a derived class's method to tell the compiler "I intend this to override a base virtual function — please verify that, and error out if it doesn't actually

match." `final` can be applied to a virtual function (no further class may override it) or to a whole class (no class may inherit from it at all).

```
class Shape {
public:
    virtual double area() const = 0;
};
class Circle : public Shape {
public:
    double area() const override { return 3.14; } // compiler-verified
    // double Area() const override; -- COMPILE ERROR: no such virtual in Shape
    // (a typo that would otherwise silently create a brand-new, unrelated
    // function instead of overriding anything -- exactly the bug override exists to catch)
};
class UnitCircle final : public Circle {};
```

COMMON MISTAKE

Misspelling or mistyping a virtual function's signature when "overriding" it without the `override` keyword doesn't error — it silently creates a new, separate, non-overriding function with a similar name. Callers through a base pointer keep calling the *base* version, and the bug can go unnoticed for a long time. Always write `override` on every intended override; there is no real downside.

CHAPTER 2 RECAP

- Inheritance: `public` is the overwhelming default; avoid it for pure code reuse
- Overriding (virtual, runtime) is unrelated to overloading (same name, compile-time)
- A virtual function is resolved at runtime via the object's actual type, not the pointer's declared type
- Missing virtual destructor + delete through base pointer = derived destructor never runs
- Always write `override` — it turns a silent bug into a compile error

03 How It Works Internally

Most tutorials stop at "virtual functions enable polymorphism." This chapter answers the question a strong interviewer actually wants answered: *how*, mechanically, does the program know which function to call? What does an object actually look like in memory? All layouts below are the *typical* approach real compilers (GCC, Clang, MSVC) use — the C++ standard itself does not mandate any specific implementation, only the observable behavior.

3.1 Object memory layout, intuitively

A non-polymorphic object is, in memory, simply its data members laid out one after another (with some compiler-inserted padding for alignment).

```
class Point {
    int x_; // 4 bytes
    int y_; // 4 bytes
};
// sizeof(Point) == 8

Point p;
+-----+-----+
|  x_  |  y_  |
+-----+-----+
offset 0  offset 4
```

When a class inherits from another, the derived object's memory contains the **base subobject** first, followed by the derived class's own new members:

```
class Animal { int age_; }; // 4 bytes
class Dog : public Animal { int breed_id_; }; // Animal's 4 + Dog's 4

Dog d;
+-----+-----+
| Animal part | Dog's own |
| (age_)      | (breed_id_) |
+-----+-----+
offset 0      offset 4
// A Dog* can be implicitly treated as an Animal* because the Animal
// subobject genuinely starts at the same address -- no conversion needed.
```

3.2 The vtable and vptr: how dynamic dispatch actually works

The moment a class has *any* virtual function, the compiler adds one hidden pointer to every object of that type: the **vptr** (virtual table pointer). It also generates, *once per class* (not per object), a **vtable** — an array of function pointers, one slot per virtual function, pointing at whichever implementation that specific class actually uses.

```
class Shape { virtual double area() const; virtual ~Shape(); };
class Circle : public Shape { double area() const override; ~Circle() override; };
```

ONE vtable per CLASS (shared by every instance of that class):

Shape's vtable:	Circle's vtable:	
+-----+	+-----+	
&Shape::area	&Circle::area	<- overridden
+-----+	+-----+	
&Shape::~Shape	&Circle::~Circle	<- overridden
+-----+	+-----+	

EVERY object gets its own vptr, set by the constructor:

```
Circle c;          c's memory:
                   +-----+
                   | vptr | radius_ (own) |
                   +-----+
                   |
                   v
                   points at Circle's vtable (above)
```

A virtual call like `shape_ptr->area()` compiles down to roughly: *follow the object's vptr to find its vtable, index into the vtable at the fixed slot reserved for `area()`, call whatever function pointer is sitting there*. This is exactly why it works correctly no matter what the pointer's *declared* type is — the vptr belongs to the actual object, not the pointer, so it always points at the vtable matching the object's *real* class.

WHY THIS DESIGN

This is also precisely why a virtual call costs slightly more than a normal call — two memory reads (object → vptr, vptr → vtable slot) plus an indirect call through a computed address, instead of the compiler jumping directly to a known address at compile time. In hot loops this is occasionally measurable, which is the honest, complete answer to "what's the cost of virtual functions" (not just "a little slower").

3.3 Static binding vs. dynamic binding

Static binding (a.k.a. early binding) means the compiler decides which function to call at compile time, based purely on the declared (static) type of the expression. **Dynamic binding** (late binding) means the decision is deferred to runtime, based on the object's actual type, via the vtable mechanism above. Non-virtual functions, overloaded functions, and non-polymorphic calls are all statically bound; virtual function calls through a pointer or reference are dynamically bound.

```
Shape s;          Shape& ref = s;
Circle c;         Shape& ref2 = c;

ref.area();       // static type Shape, dynamic type Shape -> Shape::area()
ref2.area();      // static type Shape, dynamic type Circle -> Circle::area() (dynamic binding)
c.area();         // called directly on a Circle object -- still dynamically bound,
                  // but since static and dynamic type agree here, it LOOKS static
```

3.4 Object slicing: what it is, why it happens, how to avoid it

```
void printArea(Shape s) {           // BUG: takes Shape BY VALUE, not by reference
    std::cout << s.area();
}

Circle c(5.0);
printArea(c);                       // c is SLICED down to just its Shape part
// Inside printArea, "s" is a genuine, separate Shape object -- Circle's
// radius_ member and its vtable pointer are simply not there. s.area()
// calls Shape::area(), NOT Circle::area(), even though c "is a" Circle.
```

Why this happens: passing an object by value invokes the base class's copy constructor, which only knows how to copy the base class's own members — there's no way for a plain Shape copy constructor to know about, or copy, a derived class's extra members. The object is "sliced" down to exactly the size and shape of the base class. This is a real, silent correctness bug, not a warning: the code compiles cleanly and runs, just wrong.

HOW TO AVOID IT

Pass and return polymorphic types by **reference** or **pointer** (const Shape&, Shape*, or a smart pointer), never by value. If you find yourself needing value semantics for a polymorphic hierarchy, that's usually a sign you actually want a different design (a variant, or a "cloneable" pattern), not a plain base class taken by value.

INTERVIEW NOTES

A very common gotcha question: "what happens if you store Circle objects in a std::vector<Shape>?" The answer is slicing on every insertion — a strong candidate immediately says "use std::vector<std::unique_ptr<Shape>> instead" without being prompted.

3.5 RTTI: dynamic_cast, typeid, and safer alternatives

RTTI (Run-Time Type Information) is the mechanism that lets a program query an object's actual type *at runtime*, rather than relying only on the compile-time static type. It requires the class to be polymorphic (have at least one virtual function) — the type information is stored alongside the vtable.

```
Shape* s = getRandomShape();

// dynamic_cast: safe downcasting -- returns nullptr on failure (for pointers)
if (Circle* c = dynamic_cast<Circle*>(s)) {
    // only enters here if s genuinely points at a Circle
}

// typeid: query the exact runtime type
if (typeid(*s) == typeid(Circle)) { /* ... */ }
```

Downcasting with a plain `static_cast` instead is *faster* (no runtime check) but *unsafe* — if `s` doesn't actually point at a `Circle`, `static_cast` doesn't stop you; it just silently produces a pointer that will cause undefined behavior the moment you use it.

WHY IT'S OFTEN DISCOURAGED

Needing `dynamic_cast` frequently is usually a design smell — it means code is asking "what concrete type are you?" instead of just calling a virtual function and letting dynamic dispatch (Section 3.2) answer that question implicitly. The common alternative: add a virtual function to the base interface that does what you actually need, and let each subclass implement it, rather than branching on type outside the hierarchy.

CHAPTER 3 RECAP

- A derived object's memory = base subobject, then its own new members
- One vptr per polymorphic *object*; one vtable per polymorphic *class*
- A virtual call: object → vptr → vtable slot → indirect call
- Static binding = compile-time (declared type); dynamic binding = runtime (actual type)
- Passing a derived object by value into a base-typed parameter slices it
- Heavy reliance on `dynamic_cast` often signals a missing virtual function in the interface

04 Special Member Functions & RAII

This is where OOP theory meets real, production C++ — how objects manage resources (memory, file handles, locks) correctly and automatically. If a C++ interview only has time for one deep chapter, it's usually this one.

4.1 The Rule of 0 / 3 / 5

Every class has (up to) five **special member functions** the compiler can generate for you automatically: the destructor, the copy constructor, the copy assignment operator, the move

constructor, and the move assignment operator. The Rule of 3/5 is a guideline about when you must take over writing them yourself.

RULE	SAYS	BECAUSE
Rule of 0	Write <i>none</i> of the five — let the compiler generate all of them	Your class only holds members (like <code>std::string</code> , <code>std::vector</code> , smart pointers) that already manage their own resources correctly
Rule of 3	If you write <i>any one</i> of destructor / copy constructor / copy assignment, write all three	Needing a custom one is usually a sign the class owns a raw resource (a pointer from <code>new</code>), and the compiler-generated versions of the other two will handle that resource wrong
Rule of 5	(C++11+) The same, but for all five — destructor, copy ctor, copy assign, move ctor, move assign	If you don't write the move operations, they're implicitly deleted the moment you write any of the copy/destructor ones — every "copy" silently becomes an expensive real copy instead of a cheap move

INTERVIEW NOTES

The Rule of 0 is the modern, preferred outcome — it's a signal that a class was *designed* well (delegating resource ownership to a member that already handles it), not that the author forgot something. Saying "I'd aim for Rule of 0 by using a smart pointer member" is a stronger answer than jumping straight to hand-writing all five.

4.2 Copy semantics vs. move semantics

Copying duplicates a resource — after copying, two independent objects exist, each safe to destroy on its own. **Moving** (C++11) transfers ownership of a resource from a (usually temporary, about-to-be-destroyed) source object to a new one, leaving the source in a valid-but-unspecified, safely-destructible empty state — no duplication, just a pointer handoff.

```

class Buffer {
public:
    explicit Buffer(size_t n) : size_(n), data_(new int[n]) {}

    // Copy constructor: allocate NEW storage, duplicate the contents
    Buffer(const Buffer& other) : size_(other.size_), data_(new int[other.size_]) {
        std::copy(other.data_, other.data_ + size_, data_);
    }
    // Move constructor: STEAL the pointer, leave the source empty
    Buffer(Buffer&& other) noexcept : size_(other.size_), data_(other.data_) {
        other.data_ = nullptr; // source no longer owns anything -- safe to destroy
        other.size_ = 0;
    }
    ~Buffer() { delete[] data_; }
private:
    size_t size_;
    int* data_;
};

```

The move constructor takes an **rvalue reference** (`Buffer&&`) — a reference that can only bind to a temporary or something explicitly marked movable via `std::move()`. This is the compiler's way of guaranteeing "the object I'm about to steal from is genuinely about to be discarded, so gutting it is safe."

COMMON MISTAKE

Forgetting `noexcept` on the move constructor. Standard containers like `std::vector` check at compile time whether your move constructor is `noexcept`; if it isn't, operations like resizing silently fall back to the (slower, copying) copy constructor instead, specifically because a throwing move partway through would leave the container in a corrupted, unrecoverable state — the container has no way to roll back a half-completed move.

4.3 RAII: Resource Acquisition Is Initialization

RAII is the idiom, unique in flavor to C++, of tying a resource's lifetime to an object's lifetime: acquire the resource in the constructor, release it in the destructor. Because C++ guarantees destructors run automatically when an object goes out of scope — including when an exception is thrown and the stack unwinds — RAII makes resource leaks structurally difficult, not just a matter of remembering to clean up.

```

class LockGuard {
public:
    explicit LockGuard(std::mutex& m) : mutex_(m) { mutex_.lock(); }
    ~LockGuard() { mutex_.unlock(); }    // ALWAYS runs, even if an exception is thrown in between
private:
    std::mutex& mutex_;
};

void doWork(std::mutex& m) {
    LockGuard guard(m);    // Lock acquired here
    riskyOperation();      // if this throws, guard's destructor STILL runs -- unlocked correctly
}    // Lock released here, on every exit path

```

This single idiom is why manual `lock()/unlock()`, `malloc()/free()`, or `fopen()/fclose()` pairs are considered outdated C++ style — every one of them can be forgotten on some exit path (an early return, an exception); an RAII wrapper physically cannot forget.

4.4 Smart pointers: `unique_ptr`, `shared_ptr`, `weak_ptr`

Smart pointers are RAII applied specifically to heap memory ownership — they are the modern, idiomatic replacement for raw `new/delete`.

TYPE	OWNERSHIP MODEL	COST	USE WHEN
<code>unique_ptr<T></code>	Exclusive — exactly one owner; move-only, not copyable	Zero overhead vs. a raw pointer	The default choice whenever ownership doesn't need to be shared
<code>shared_ptr<T></code>	Shared — reference-counted; the object is destroyed when the last owner goes away	Atomic refcount increment/decrement on copy/destroy	Genuinely multiple, simultaneous owners with no clear single one
<code>weak_ptr<T></code>	Non-owning observer of a <code>shared_ptr</code> -managed object; doesn't affect the refcount	Small, but must be "locked" (<code>.lock()</code>) into a temporary <code>shared_ptr</code> to actually use	Breaking a reference cycle (Section 10), or observing an object without keeping it alive

```

auto owner = std::make_unique<Circle>(5.0);    // sole owner
// auto copy = owner;    -- COMPILE ERROR: unique_ptr cannot be copied
auto moved = std::move(owner);                // ownership transferred; owner is now nullptr

auto shared1 = std::make_shared<Circle>(5.0);    // refcount == 1
auto shared2 = shared1;                        // refcount == 2, both point at the same object

```

WHY PREFER `MAKE_UNIQUE/MAKE_SHARED`

`std::make_unique<T>(...)` and `std::make_shared<T>(...)` are preferred over `unique_ptr<T>(new T(...))` for two real reasons: they avoid writing the type name twice, and — for `shared_ptr` specifically

— `make_shared` allocates the object and its control block in *one* heap allocation instead of two, which is both faster and safer under certain exception-throwing edge cases during construction.

4.5 Deep copy vs. shallow copy

A **shallow copy** duplicates only the top-level members — if a member is a raw pointer, the pointer's *value* (the address) is copied, not what it points to, leaving both the original and the copy pointing at the same underlying data. A **deep copy** duplicates the pointed-to data itself, producing two fully independent objects. The compiler-generated copy constructor always does a shallow, member-wise copy — which is exactly correct for members that manage their own deep-copying (like `std::string`), and exactly wrong for a raw owning pointer (Section 4.2's Buffer example hand-writes a deep copy for precisely this reason).

COMMON MISTAKE

Relying on the compiler-generated (shallow) copy constructor for a class holding a raw owning pointer. Two objects end up believing they each own the same memory; when both destructors run, the same pointer is deleted twice — a **double free**, undefined behavior, and a classic source of hard-to-reproduce crashes.

4.6 Exception safety guarantees in class design

GUARANTEE	PROMISE
Basic	If an exception is thrown, no resources leak and the object remains in <i>some</i> valid (but possibly changed) state
Strong	If an exception is thrown, the operation has no effect at all — it's as if it was never called (often implemented via copy-then-swap)
No-throw (<code>noexcept</code>)	The operation is guaranteed never to throw at all

RAII (Section 4.3) is what makes even the *basic* guarantee achievable by default in C++ — because destructors run automatically during stack unwinding, resources don't leak just because an exception passed through. Move constructors are conventionally marked `noexcept` whenever genuinely possible (Section 4.2) specifically because standard containers rely on that guarantee to choose the faster, move-based code path safely.

CHAPTER 4 RECAP

- Rule of 0 (preferred) / 3 / 5 — write all-or-none of the special member functions
- Copy duplicates a resource; move steals it, leaving the source empty but valid

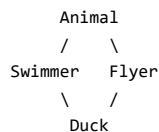
- RAll: tie a resource's lifetime to an object's lifetime via constructor/destructor
- `unique_ptr` by default; `shared_ptr` only for genuine shared ownership; `weak_ptr` to break cycles
- A raw owning pointer + the default (shallow) copy constructor = double free waiting to happen
- Mark move constructors `noexcept` whenever truly non-throwing — containers depend on it

05 Advanced Inheritance

Multiple inheritance, when it's genuinely useful, when it creates the notorious diamond problem, and the decision framework for choosing inheritance vs. composition in the first place.

5.1 Multiple inheritance and the diamond problem

C++, unlike many OOP languages, allows a class to inherit from more than one base class directly. This becomes a real problem when two base classes themselves share a common ancestor:



If Swimmer and Flyer each inherit their own copy of Animal, Duck ends up with TWO separate Animal subobjects -- ambiguous which one "d.age()" (a member of Animal) should refer to.

```

class Animal { public: int age_; };
class Swimmer : public Animal {};
class Flyer   : public Animal {};
class Duck : public Swimmer, public Flyer {};

Duck d;
// d.age_ = 3;  -- COMPILE ERROR: ambiguous -- Swimmer::age_ or Flyer::age_?
d.Swimmer::age_ = 3;  // must disambiguate explicitly

```

5.2 Virtual inheritance: the fix

```

class Animal { public: int age_; };
class Swimmer : virtual public Animal {}; // virtual here
class Flyer   : virtual public Animal {}; // and here
class Duck : public Swimmer, public Flyer {};

Duck d;
d.age_ = 3; // fine now -- exactly ONE shared Animal subobject

```

virtual inheritance tells the compiler that all classes inheriting virtually from the same base should share exactly *one* instance of it, no matter how many separate paths lead to it. The mechanism costs something real: the shared base subobject's exact location can't be known at compile time anymore (it depends on the full, final most-derived class), so accessing it requires an extra indirection through a hidden pointer — similar in spirit to the vptr from Chapter 3.

COMMON MISTAKE

Reaching for virtual inheritance defensively, "just in case," on every base class. It adds real runtime and memory cost, and is only actually needed at the exact point a diamond shape like the one above would otherwise occur — most C++ codebases go years without needing it at all.

5.3 Composition vs. inheritance — the decision rule

Composition means a class holds another class as a member ("has-a") rather than inheriting from it ("is-a"). The standard, widely-taught guideline is: **favor composition over inheritance** by default.

```

// Inheritance: is a Car genuinely a kind of Engine? No.
class Car : public Engine {}; // wrong relationship -- reuse via the wrong tool

// Composition: a Car HAS an Engine. Correct.
class Car {
private:
    Engine engine_; // composition -- Car delegates to engine_, doesn't pretend to BE one
};

```

QUESTION TO ASK	ANSWER FAVORS
Is the relationship genuinely "is-a," substitutable everywhere the base is expected?	Inheritance
Do you just want to reuse some code, with no real is-a relationship?	Composition
Does the "sub" type need to change its held object at runtime?	Composition (swap the member)
Would inheriting break Liskov Substitution (a subclass that can't honor the base's contract)?	Composition

Inheritance couples a derived class tightly to its base's implementation details — a change to the base can ripple into every subclass in ways that are hard to predict (the "fragile base class" problem). Composition keeps that coupling to an explicit, narrow interface, which is precisely why it's the safer default.

5.4 Interfaces vs. implementation inheritance

Interface inheritance means inheriting only a contract (pure virtual functions, Section 2.4) with no shared implementation — the base class defines *what* must be done, each derived class defines *how*. **Implementation inheritance** means inheriting actual, shared code (non-pure virtual or ordinary member functions) to avoid duplicating it. The first is almost always safe and is what "favor composition over inheritance" is *not* arguing against — a pure interface has none of the fragile-base-class coupling problems, since there's no shared implementation to become entangled with.

INTERVIEW NOTES

A precise answer distinguishes these two explicitly: "I'd use interface inheritance freely — it's just a contract. I'd be much more cautious about implementation inheritance, and lean toward composition there instead." This single distinction is often what separates a memorized "favor composition" answer from one that demonstrates real understanding of *why*.

CHAPTER 5 RECAP

- Multiple inheritance can create a diamond: two copies of a shared ancestor
- virtual inheritance collapses the diamond to one shared subobject, at a real cost
- Favor composition over inheritance by default — ask "is-a" honestly, not "can I reuse this"
- Interface inheritance (pure contracts) is much safer than implementation inheritance (shared code)

06 Operator Overloading

C++ lets you give operators like `+`, `==`, and `<<` a meaning for your own types. Used well, this makes code read naturally; used badly, it makes code actively deceptive — this chapter covers both halves.

6.1 What to overload, and what not to

The guiding principle is the **principle of least astonishment**: an overloaded operator should behave the way a reader would naturally expect from that symbol's meaning on built-in types. `+` on two `Money` objects should add them; it should never, say, secretly log a transaction to a file.

GOOD CANDIDATES	POOR CANDIDATES
Arithmetic on a numeric-like type (<code>Vector2D + Vector2D</code>)	<code>+</code> meaning something unrelated to addition
Comparison (<code>==</code> , <code><</code>) for value-like types	Overloading <code>&&/ </code> — loses short-circuit evaluation entirely
Stream insertion (<code><<</code>) for printing/logging	Overloading <code>,</code> (comma) — almost always more confusing than helpful
Function-call operator (<code>()</code>) for a "functor" type	Any operator whose overloaded meaning surprises a reader familiar with the built-in behavior

6.2 Assignment (`operator=`)

```
class Buffer {
public:
    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this; // guard against self-assignment: buf = buf;
        delete[] data_;
        size_ = other.size_;
        data_ = new int[size_];
        std::copy(other.data_, other.data_ + size_, data_);
        return *this; // enables chaining: a = b = c;
    }
private:
    size_t size_; int* data_;
};
```

Two non-obvious requirements: check for **self-assignment** (`x = x`; is legal C++ and must not corrupt the object — without the guard above, `delete[] data_` would free memory that `other.data_` also still points at, right before trying to read from it), and **return `*this` by reference** to support chained assignment.

6.3 Equality (operator==) and the spaceship operator (<=>)

```
// Traditional (any standard): write == explicitly, then != in terms of it
bool operator==(const Point& a, const Point& b) {
    return a.x == b.x && a.y == b.y;
}
bool operator!=(const Point& a, const Point& b) { return !(a == b); }

// C++20: the spaceship operator generates ==, !=, <, <=, >, >= all at once
class Point {
public:
    auto operator<=>(const Point&) const = default; // member-wise comparison, all six operators
private:
    int x, y;
};
```

The **spaceship operator** (<=>, C++20) is a genuine simplification: instead of hand-writing up to six comparison operators (and keeping them all consistent with each other by hand), you write *one* — the compiler generates the rest from it. Under C++17, you still write each comparison operator individually, conventionally implementing the others in terms of the smallest necessary set (== and <, with the rest derived).

6.4 Stream operators (<<, >>)

```
// Must be a free function (or friend, Chapter 8) -- NOT a member --
// because the left-hand operand is std::ostream, not your class.
std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}
// Now: std::cout << myPoint; works naturally
```

COMMON MISTAKE

Trying to write operator<< as a member function. A member operator's implicit first (left-hand) parameter is always the object it's called on — but in std::cout << p, the left-hand side is std::cout, a type you don't own and can't add members to. It must be a free function that takes both operands explicitly.

6.5 General pitfalls

- **Returning by value when a reference would do** — for compound-assignment operators like +=, return *this by reference, not a new object by value.
- **Asymmetric overloads** — defining Money + int but not int + Money (the free-function form, not a member, is what makes both orders possible).

- **Overloading purely for cleverness** — if a reader has to stop and think about what an operator does for your type, it has failed the principle of least astonishment, regardless of how elegant the implementation is.

CHAPTER 6 RECAP

- Overload only when the meaning matches what the symbol already suggests
- `operator=` needs a self-assignment guard and must return `*this` by reference
- C++20's `<=>` generates all six comparisons from one function
- Stream operators must be free functions — the left-hand type isn't yours to add members to

07 Const-Correctness & API Design

A class's `const` annotations are a promise, checked by the compiler, about what a piece of code is and isn't allowed to change. Getting this right is what separates an API that's pleasant to use from one where every caller has to guess.

7.1 `const` methods

```
class Circle {
public:
    double area() const { return 3.14159 * radius_ * radius_; } // promises: won't modify *this
    void setRadius(double r) { radius_ = r; }                  // no const -- it DOES modify
private:
    double radius_;
};
```

A `const` after a member function's parameter list promises the compiler (and every caller) that calling this method will not modify the object's observable state. The compiler enforces this: attempting to assign to a member inside a `const` method is a compile error.

7.2 `Const` objects and `const` references

```
void printArea(const Circle& c) { // promises: this function won't modify c
    std::cout << c.area();      // fine -- area() is a const method
    // c.setRadius(10);        -- COMPILE ERROR: can't call a non-const
    //                          method on a const reference
}

const Circle unitCircle(1.0);
// unitCircle.setRadius(2.0);  -- COMPILE ERROR: object itself is const
```

Passing by `const&` is the default, idiomatic way to pass a non-trivial object into a function that only needs to read it — it avoids an unnecessary copy (unlike pass-by-value) while still preventing the function from mutating the caller's object (unlike a plain, non-`const` `&`).

WHY THIS MATTERS FOR API DESIGN

A function signature that takes `const Circle&` is, itself, documentation — a caller can tell at a glance, without reading the implementation, that their object is safe from modification. A signature that takes a plain `Circle&` when it doesn't need to mutate anything actively misleads callers into unnecessary defensive copying.

7.3 `mutable`, and when it's justified

```
class ExpensiveComputation {
public:
    int result() const {
        if (!cached_) {
            cache_ = computeExpensively(); // mutating a "const" method -- legal because cache_ is mutable
            cached_ = true;
        }
        return cache_;
    }
private:
    mutable int cache_ = 0; // implementation detail: caching, not observable state
    mutable bool cached_ = false;
};
```

`mutable` lets a specific member be modified even from inside a `const` method. The justification has to be precise: the member is an **implementation detail** (a cache, a mutex used for internal thread-safety, a lazily-computed value) that doesn't affect the object's *logical*, externally-observable state — `result()` genuinely behaves as "const" from a caller's point of view, always returning the same answer for the same inputs, even though its internals mutate on first use.

COMMON MISTAKE

Using `mutable` as an escape hatch to silence a const-correctness compile error on a member that *is* real, observable state. That defeats the entire purpose of marking the method `const` in the first place — it's lying to every caller who trusted the promise.

7.4 Designing clean class APIs

- **Mark every method `const` that can honestly be** — it's not just documentation, it's what lets callers hold `const` references/objects and still call your methods (Section 7.2).

- **Prefer passing by `const&`** for anything larger than a couple of machine words, unless you specifically need a copy (then take by value and move, per Chapter 4) or need to mutate the caller's object (then a plain `&`, and name it clearly).
- **Keep the public interface minimal** — every public method is a promise you'll support indefinitely; prefer private/protected by default and widen access only when a real caller needs it (a direct application of encapsulation, Section 1.5).
- **Make invalid states unrepresentable where possible** — a constructor that validates its inputs and a class that maintains its invariant through every public method is a class whose users can't accidentally misuse it.

CHAPTER 7 RECAP

- `const` on a method is a compiler-checked promise not to modify the object
- `mutable` is only justified for implementation details invisible to the object's logical state
- `const&` parameters avoid copies while still protecting the caller's object
- A clean API: const-correct where possible, minimal public surface, invariants enforced by construction

08 Static Members, Friends & Other Tools

A handful of C++-specific tools that don't fit neatly into "the four pillars" but come up constantly in real code and in interviews.

8.1 Static data members

```
class Widget {
public:
    Widget() { ++count_; }
    ~Widget() { --count_; }
    static int count() { return count_; } // static member FUNCTION, see 8.2
private:
    static inline int count_ = 0; // ONE shared counter, not per-object (C++17: inline avoids
                                // a separate out-of-class definition)
};

Widget a, b, c;
Widget::count(); // 3 -- shared across every instance, accessed via the class, not an object
```

A **static data member** belongs to the *class itself*, not to any individual object — there is exactly one copy, shared by every instance, existing even if zero objects have been created. Before C++17, a static member needed a separate definition outside the class (in one `.cpp` file); `inline` (C++17) lets you define and initialize it directly in the class body.

8.2 Static member functions

A **static member function** (like `count()` above) has no implicit `this` pointer — it can't access non-static members at all, and is called through the class name (`Widget::count()`) rather than through an object. It's the right tool for a function that logically belongs to the class (a factory function, a utility tied to the type) but doesn't need any particular object's state.

8.3 friend functions and classes

```
class Matrix {
public:
    friend Matrix operator+(const Matrix& a, const Matrix& b); // grants access
private:
    double data_[4][4];
};

Matrix operator+(const Matrix& a, const Matrix& b) {
    Matrix result;
    // can read a.data_ and b.data_ directly here -- normally private/inaccessible
    return result;
}
```

`friend` grants a specific external function or class access to a class's private/protected members, bypassing encapsulation (Section 1.5) in a deliberate, narrow, named way. It's the standard tool for the stream-operator problem from Section 6.4: when `operator<<` must be a free function but genuinely needs access to private internals to print them.

USE CASE	RISK
A free-function operator that needs private access (<code><<</code> , some arithmetic operators)	Low — narrow, well-understood pattern
Tightly coupled helper classes (e.g. a class and its dedicated iterator)	Moderate — can indicate the two should actually be one class, or need a better public interface
Reaching for <code>friend</code> broadly to avoid designing a proper public interface	High — a direct encapsulation violation; the class's invariant (Section 1.5) is no longer enforced solely by the class itself

INTERVIEW NOTES

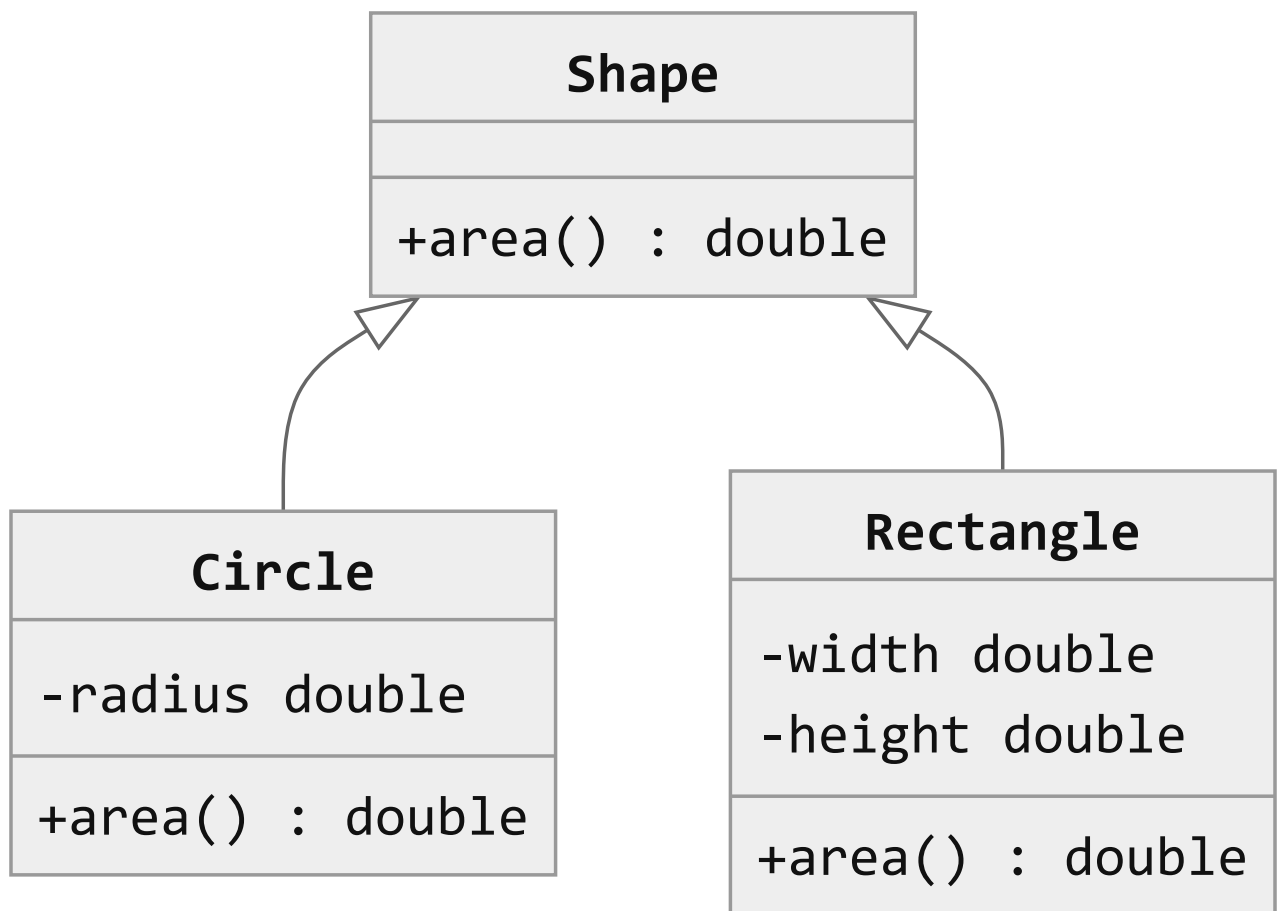
Friendship in C++ is **not inherited and not mutual** — a friend of a base class is not automatically a friend of a derived class, and if class A declares class B a friend, B does not automatically consider A a friend back. Both of these are commonly tested, easy-to-get-wrong details.

- Static data members: one copy, shared across all objects, exists independent of instances
- Static member functions: no `this`, called via the class name, can't touch non-static members
- friend grants deliberate, narrow access — overusing it undermines encapsulation
- Friendship is neither inherited nor mutual

09 Diagrams & Mental Models

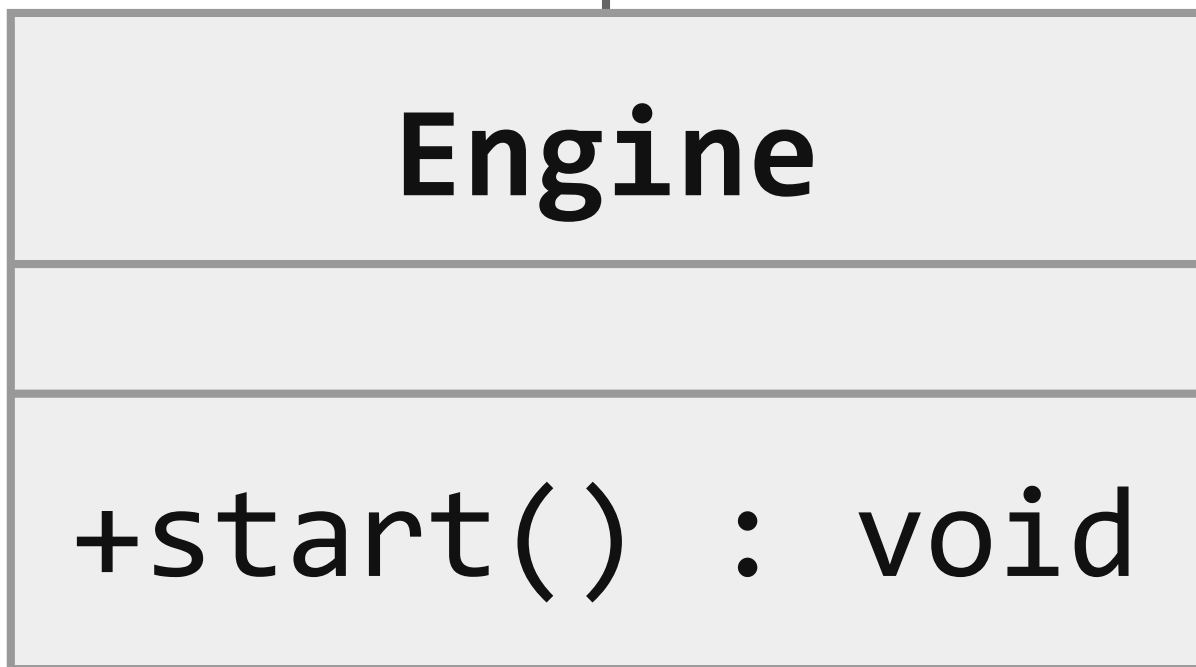
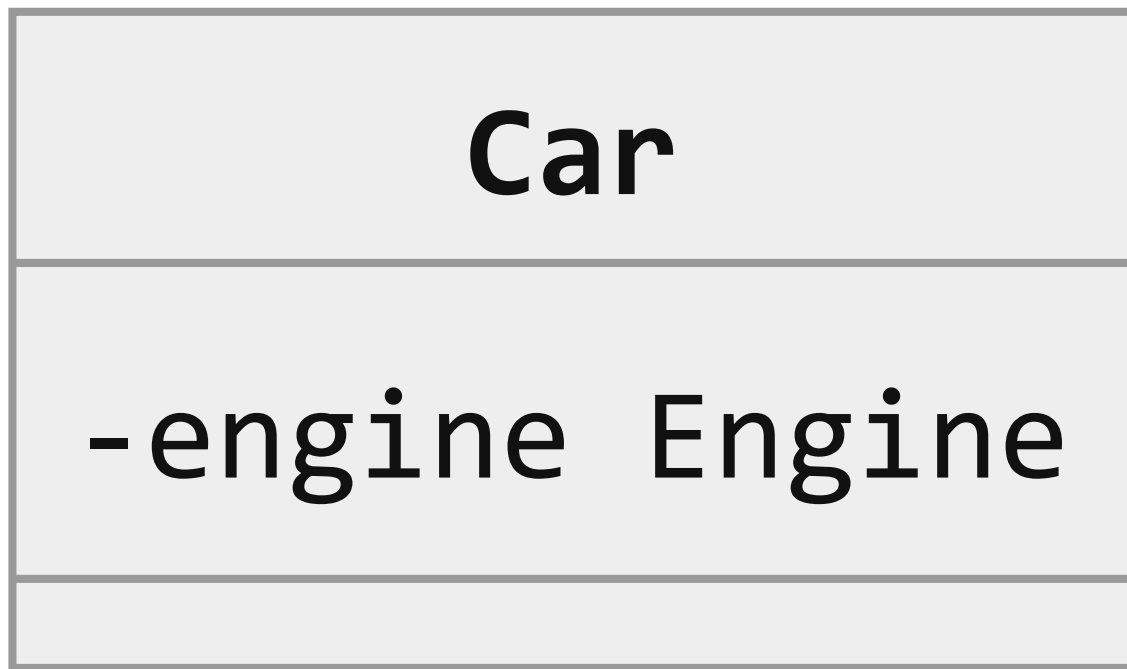
The four class relationships used throughout this book, drawn side by side, so you can tell them apart at a glance on a whiteboard.

9.1 Inheritance ("is-a")



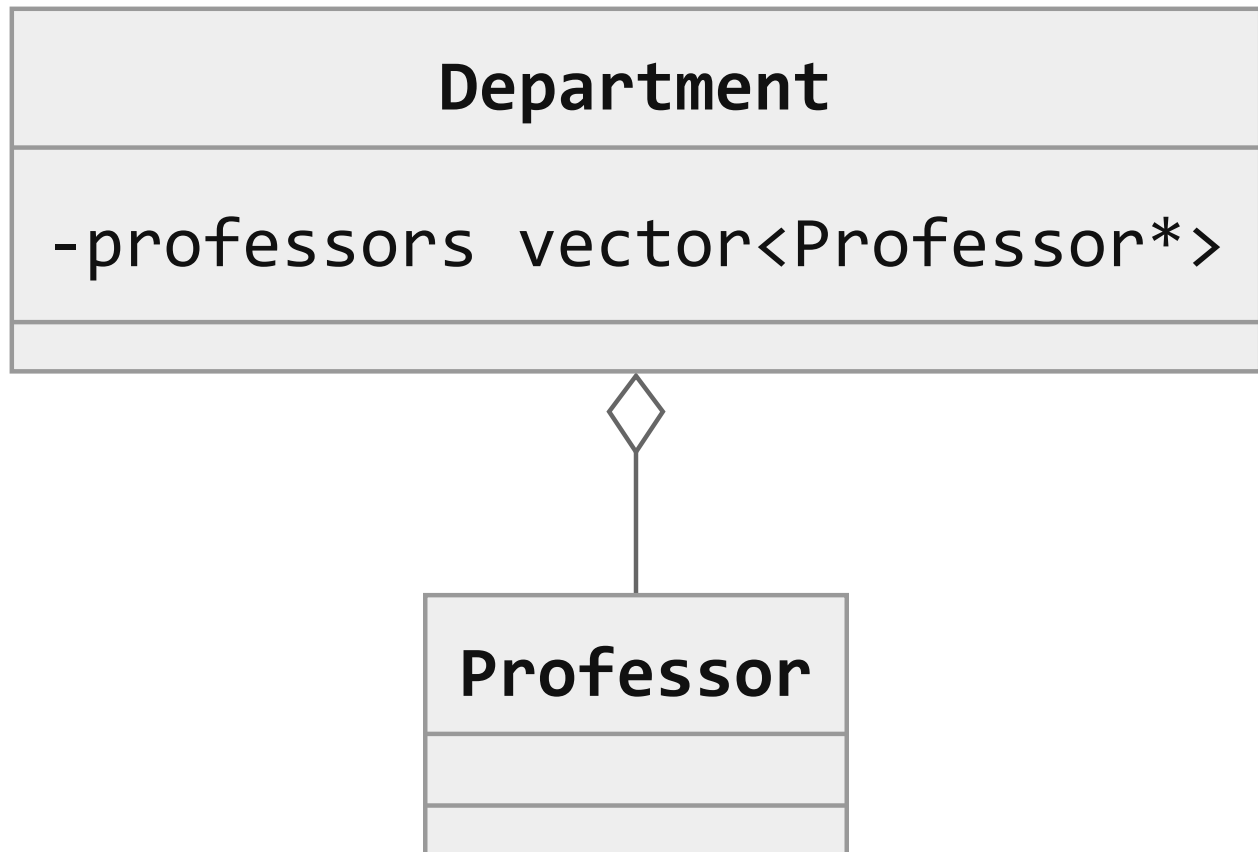
The hollow-triangle arrow points from the derived class to the base class. **Circle** and **Rectangle** each override `area()` (Chapter 2). This is the relationship diagram for everything in Chapters 2, 3, and 5.

9.2 Composition ("owns-a," lifetime-bound)



The filled diamond means **composition**: Engine's lifetime is entirely owned by Car — destroy the Car, its Engine member is destroyed with it (Section 5.3). This is the relationship RAII and the Rule of 0 (Chapter 4) are built around.

9.3 Aggregation ("has-a," independent lifetime)



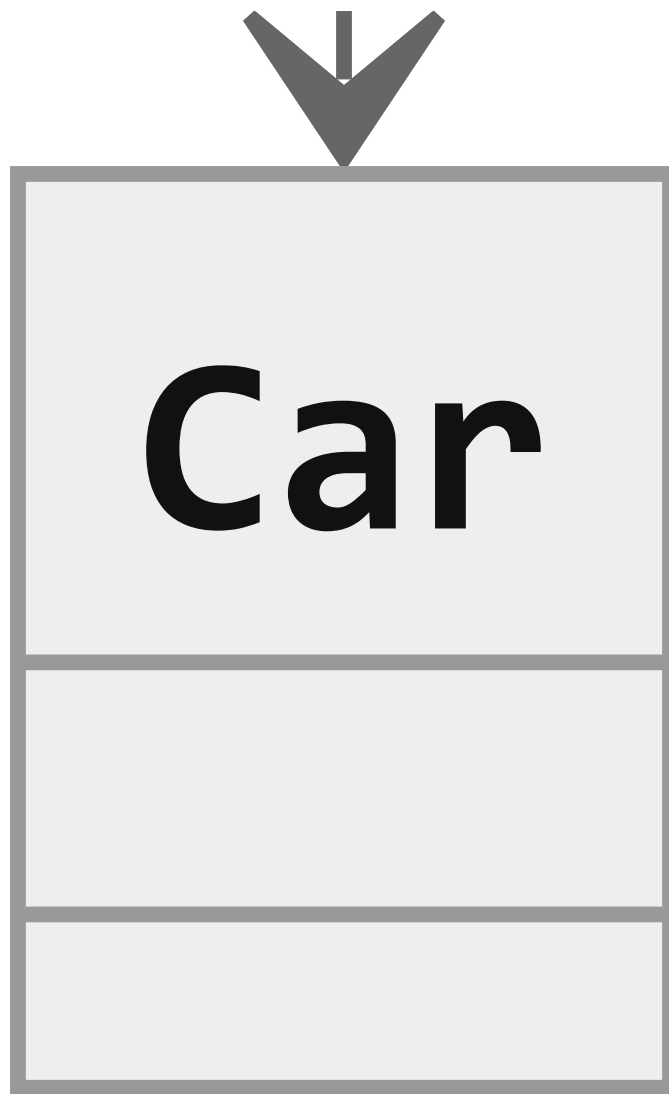
The hollow diamond means **aggregation**: a **Department** holds references/pointers to **Professors**, but a **Professor** can exist independently and outlive the **Department** (or move to a different one). The distinction from composition is entirely about *lifetime ownership*, not the syntax used to hold the reference.

9.4 Association (a general "uses")

Driver

The diagram consists of a large rectangular box at the top, divided into three horizontal sections. The top section contains the word 'Driver' in a bold, black, sans-serif font. The middle and bottom sections are empty. A vertical line extends from the bottom center of this box to the top center of a smaller rectangular box below it. This smaller box contains the word 'drives' in a black, sans-serif font.

drives



A plain line means **association** — the most general relationship, just "these two types interact," with no ownership implied in either direction. A function parameter, a method that briefly uses another object without storing it, are typically just associations.

9.5 All four, compared

RELATIONSHIP	MEANING	LIFETIME COUPLING
Inheritance	"Is-a" — substitutable for the base	Derived object <i>contains</i> the base subobject
Composition	"Owns-a," part cannot outlive the whole	Tightly coupled — same lifetime
Aggregation	"Has-a," part can outlive the whole	Loosely coupled — independent lifetimes
Association	"Uses-a," general interaction	No ownership implied at all

CHAPTER 9 RECAP

- Inheritance: hollow triangle, is-a, substitutable
- Composition: filled diamond, owns-a, same lifetime
- Aggregation: hollow diamond, has-a, independent lifetime
- Association: plain line, uses-a, no ownership

10 Interview Question Bank

Every question here builds on a chapter above — cross-references are given so you can go back and re-read the mechanics if an answer doesn't fully click. Several questions cover ground the chapters above set up but didn't fully close (copy elision, `shared_ptr` cycles, virtual calls inside constructors, CRTP, PIMPL) — read this chapter in full, not just as a quiz.

Basic

Q1 **BASIC** What's the actual difference between `class` and `struct` in C++?

Exactly one default: members and inheritance default to `public` in a `struct`, and `private` in a `class` (Section 1.1). Everything else — constructors, virtual functions, templates, inheritance — works identically on both. Idiomatic C++ still uses the two as a style signal: `struct` for a passive bundle of public data with no invariants to protect, `class` for a type that enforces real invariants (Section 1.5).

What's being tested: whether you know the difference is purely about defaults, not a real language-level distinction.

Q2 BASIC What does this print, and in what order?

```
class Base { public: Base() { std::cout << "Base ctor\n"; } ~Base() { std::cout << "Base dtor\n"; } };
class Derived : public Base { public: Derived() { std::cout << "Derived ctor\n"; } ~Derived() { std::cout <<
"Derived dtor\n"; } };

{ Derived d; }
```

Answer: Base ctor, Derived ctor, Derived dtor, Base dtor. Construction always runs base-first, then derived (a derived object can't exist without its base part existing first — Section 3.1's memory layout literally requires the base subobject to be there). Destruction is the exact mirror: derived-first, then base — undoing construction in reverse order, the same discipline a stack unwinds in.

What's being tested: whether "base first, derived last, destruction in reverse" is automatic recall, since it underlies most of Chapters 2–3.

Q3 BASIC Overloading vs. overriding — what's the precise difference?

Overloading: multiple functions, same name, different parameter lists, resolved by the compiler at compile time (Section 2.2) — has nothing to do with inheritance. **Overriding:** a derived class replacing a base class's virtual function, same signature, resolved at runtime via the vtable (Section 3.2). A common trap: changing a parameter type while "overriding" doesn't override at all — it silently overloads instead, which is exactly why `override` (Section 2.6) should always be written explicitly.

Follow-up: "can you overload a virtual function?" — yes, freely; overloading and virtuality are unrelated axes.

Q4 BASIC Can a constructor be virtual? Why or why not?

No. A virtual call is dispatched via an object's vptr (Section 3.2) — but the vptr doesn't exist yet *until* the constructor has run enough to set it up. There is no object, and no vtable to consult, before construction completes; "virtual construction" is a contradiction in terms. (Destructors, by contrast, genuinely can and usually should be virtual, Section 2.5 — by the time a destructor runs, the object and its vptr already exist.)

What's being tested: whether you understand the vptr's setup timing, not just that "constructors can't be virtual" as a memorized fact.

Q5 BASIC What is the `this` pointer?

Inside any non-static member function, `this` is an implicit pointer to the object the function was called on — `this->memberName` and `memberName` refer to the same thing inside a member function. It's why `operator=` (Section 6.2) can return `*this` to enable chaining, and why static member functions (Section 8.2), which have no associated object, don't have a `this` at all.

Q6 BASIC What happens if you don't write any constructor at all?

The compiler generates a default constructor for you, which default-constructs every member in turn (for a raw `int` or `double` member with no in-class initializer, this leaves it **uninitialized garbage**, not zero — a very common source of bugs). It also generates the other four special member functions per the Rule of 0 discussion in Section 4.1, *unless* you've written any of them yourself, which suppresses some of the others (a classic follow-up: writing a custom destructor implicitly deletes the move constructor/assignment).

Q7 BASIC What is the difference between an abstract class and an interface in C++?

C++ has no separate `interface` keyword — the language expresses "interface" as an idiom: an abstract class (Section 1.6, 2.4) whose *every* method is pure virtual, with no data members and no shared implementation at all. An "ordinary" abstract class may mix pure virtual functions with regular ones and with data members. The distinction is purely conventional, not enforced by the compiler.

Intermediate

Q8

INTERMEDIATE

Trace through what goes wrong here, precisely.

```
class Base { public: ~Base() {} }; // not virtual
class Derived : public Base { public: ~Derived() { delete[] buf_; } private: int* buf_ = new int[10]; };

Base* p = new Derived();
delete p;
```

Full treatment in Section 2.5. Short version for an interview: `delete p` only runs `~Base()`, because destructor selection here is statically resolved from `p`'s declared type (`Base*`) — there's no vtable entry to consult without `virtual`. `~Derived()` never runs, `buf_` leaks, and this is technically undefined behavior per the standard, not merely "a leak."

Typical follow-up: "how would you catch this at compile time instead of relying on remembering it?" — a strong answer mentions static analyzers/linters (e.g. clang-tidy's relevant checks) that flag a class with virtual functions but a non-virtual destructor.

Q9

INTERMEDIATE

What's wrong with this, and what actually happens?

```
std::vector<Shape> shapes;
shapes.push_back(Circle(5.0));
shapes.push_back(Rectangle(3.0, 4.0));
for (auto& s : shapes) std::cout << s.area();
```

Full treatment in Section 3.4. Each `push_back` **slices** the argument down to a plain `Shape` — the vector never actually holds a `Circle` or a `Rectangle`, just their `Shape` parts, all calling `Shape::area()`, likely returning 0 for every element regardless of what was pushed. Fix:

`std::vector<std::unique_ptr<Shape>>`, preserving polymorphism through pointer indirection.

Q10

INTERMEDIATE

What is copy elision, and what's the difference between C++17 and earlier standards here?

```
Widget makeWidget() {  
    return Widget(42);    // constructing a temporary, returned by value  
}  
  
Widget w = makeWidget();
```

Copy elision is the compiler skipping an intermediate copy/move entirely, constructing the object directly in its final destination instead. In the example above, a naive mental model expects: construct a temporary inside `makeWidget`, move/copy it into the return slot, then move/copy it again into `w` — up to two extra constructor calls beyond the one that actually builds the 42. This specific pattern is called **RVO** (Return Value Optimization, or more precisely NRVO for a named local variable).

Before C++17, compilers were *permitted* to elide these copies (and virtually all real compilers did, in this exact pattern) but were not *required* to — your class still needed an accessible copy or move constructor, even if it was never actually called. **C++17 made this specific case (a prvalue returned and immediately used to initialize a new object) mandatory** — no copy or move constructor call happens at all, not even conceptually; the language guarantees it, and your class doesn't even need a copy/move constructor to be accessible for this exact pattern to compile.

INTERVIEW NOTES

A precise answer distinguishes "the compiler is allowed to" (pre-C++17, an optimization) from "the compiler is required to" (C++17, part of the language's value semantics) — conflating the two is the most common way this answer falls short.

Q11

INTERMEDIATE

When exactly do you need to write all five Rule-of-5 functions yourself?

Full treatment in Section 4.1. The precise trigger: your class directly manages a resource the compiler doesn't already know how to copy/move/destroy correctly on its own — almost always a raw owning pointer, a raw file handle, or a raw OS handle (socket, mutex). If every member is itself a well-behaved type (a `std::string`, a `std::vector`, a smart pointer), you're in Rule-of-0 territory and should write none of the five.

Q12

INTERMEDIATE

Draw the diamond problem and explain the fix.

Full treatment with diagrams in Section 5.1–5.2. Short version: two base classes that both (non-virtually) inherit from a common ancestor cause a most-derived class to contain two separate copies of that ancestor, making any reference to its members ambiguous. `virtual` inheritance on the two middle classes collapses them back to one shared subobject, at the cost of an extra indirection to locate it (since its offset can no longer be fixed at compile time).

Q13

INTERMEDIATE

Can a pure virtual function have a function body? Why would you ever do that?

```
class Shape {
public:
    virtual ~Shape() = 0;    // pure virtual, but WITH a body below
};
Shape::~~Shape() { std::cout << "Shape destroyed\n"; }    // legal and common
```

Yes — `= 0` only means "derived classes must override this; you cannot instantiate this class as-is." It does not forbid providing an implementation that derived classes can still explicitly invoke, most commonly for a base destructor: making the destructor pure virtual forces the class to be abstract (Section 2.4) even if there's otherwise nothing else to make it abstract, while the body still runs correctly as part of every derived object's destruction sequence (Q2's chain).

Q14

INTERMEDIATE

What is a `shared_ptr` reference cycle, and how do you break it?

```
class Node {
public:
    std::shared_ptr<Node> next;
    std::shared_ptr<Node> prev;    // BUG: should be weak_ptr
};

auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b's refcount: 2 (b itself + a->next)
b->prev = a;    // a's refcount: 2 (a itself + b->prev)
// a and b go out of scope: each refcount drops to 1, never to 0.
// Neither Node is ever destroyed. This is a real memory leak.
```

`shared_ptr` uses reference counting (Section 4.4) — an object is destroyed only when its count reaches zero. Two objects holding `shared_ptr`s to each other keep each other's count from ever reaching zero, even once nothing external references either of them — a genuine, silent memory leak, not undefined behavior (unlike most pointer bugs, the program doesn't crash; it just never frees the memory).

Fix: make one direction of the relationship a `weak_ptr` instead — typically the "back" pointer in a parent/child or doubly-linked structure (`prev` above). A `weak_ptr` doesn't contribute to the refcount, so it can observe the object without keeping it alive; accessing it requires `.lock()`, which returns a temporary `shared_ptr` (or an empty one, if the object has since actually been destroyed).

Q15

INTERMEDIATE

Why can a base pointer call a derived class's overridden method, but not a method that only exists in the derived class?

```
Base* b = new Derived();
b->virtualMethod();    // fine -- dispatched dynamically to Derived's override (Section 3.2/3.3)
// b->derivedOnlyMethod(); -- COMPILE ERROR: Base has no such member
```

Member access is checked against the **static (compile-time) type** of the expression — the compiler only knows `b` is a `Base*`, and `Base` genuinely has no `derivedOnlyMethod`. Dynamic dispatch (Section 3.2) only changes *which implementation runs* for a method that exists in the base's interface — it doesn't retroactively add new methods to that interface. Accessing a derived-only method requires an explicit downcast (`dynamic_cast`, Section 3.5) first.

Q16

INTERMEDIATE

Is `const_cast` ever legitimate to use?

Rarely, and narrowly: the one broadly accepted use is inside a `const` overload of a member function that calls its own non-`const` overload to avoid duplicating logic — casting away constness on `this` is safe there specifically because the object genuinely isn't `const` in that context (you're just working around C++'s lack of a cleaner way to share code between the two overloads). Using `const_cast` to actually mutate an object that was originally declared `const` is undefined behavior — the cast only removes the compiler's const-checking, it does not retroactively make a genuinely `const` object mutable. `mutable` (Section 7.3) is almost always the better tool for the "I need to change this from a `const` method" problem.

Advanced

What actually happens if you call a virtual function from inside a constructor or destructor?

```
class Base {
public:
    Base() { init(); }           // calling a virtual function from the CONSTRUCTOR
    virtual void init() { std::cout << "Base::init\n"; }
};
class Derived : public Base {
public:
    void init() override { std::cout << "Derived::init\n"; }
};

Derived d; // prints "Base::init" -- NOT "Derived::init"!
```

This surprises almost everyone the first time. During Base's constructor, the object *genuinely is only a Base so far* — the Derived part hasn't been constructed yet, and the object's vptr (Section 3.2) is still pointing at Base's vtable, not Derived's. C++ deliberately does *not* pretend the object is already its final derived type during construction — calling `init()` genuinely calls `Base::init()`, exactly matching the object's real, current type at that moment. The exact mirror happens in reverse during destruction: once `~Derived()` has run, the vptr is reset back to Base's vtable before `~Base()` runs, so a virtual call inside `~Base()` also resolves to Base's version, not Derived's (which may already be partially torn down anyway).

INTERVIEW NOTES

This is one of the sharpest "do you actually understand the vtable, or did you memorize the syllabus" questions available — a memorized answer says "it calls the base version," a strong answer explains *why* in terms of the vptr not yet (or no longer) pointing at the derived vtable during construction/destruction.

What is CRTP, and how does it give you "virtual-like" behavior with zero runtime cost?

```
template<typename Derived>
class Shape {
public:
    double area() const {
        return static_cast<const Derived*>(this)->areaImpl();    // "dispatch" via a cast, not a vtable
    }
};

class Circle : public Shape<Circle> {    // note: Circle inherits from Shape<Circle> -- "curiously recurring"
public:
    double areaImpl() const { return 3.14159 * r_ * r_; }
private:
    double r_ = 1.0;
};
```

The **Curiously Recurring Template Pattern** has a derived class inherit from a template instantiated *with itself* as the type argument. Because the base template now knows the exact derived type at *compile* time, it can `static_cast` this to that concrete type and call its method directly — a normal, statically-resolved function call, with no vptr, no vtable, no runtime indirection at all (Section 2.2's compile-time polymorphism, applied to something that looks like it should need runtime polymorphism). The trade-off: every distinct `Derived` produces its own separate instantiation of `Shape<Derived>`, so you lose the ability to hold a single, uniform collection of "any shape" through one common runtime base pointer — CRTP is compile-time polymorphism, not a drop-in replacement for genuine runtime polymorphism.

```
// Widget.h -- the public header, included by every caller
class Widget {
public:
    Widget();
    ~Widget();                // must be declared here, DEFINED in the .cpp (see below)
    void doSomething();
private:
    class Impl;               // forward declaration only -- callers never see its definition
    std::unique_ptr<Impl> pimpl_; // "pointer to implementation"
};

// Widget.cpp -- Impl's real definition lives here, invisible to callers
class Widget::Impl {
public:
    /* all the actual private data and helper logic */
};
Widget::Widget() : pimpl_(std::make_unique<Impl>()) {}
Widget::~Widget() = default; // defined here, where Impl is a complete type
```

PIMPL (Pointer to IMPLementation) moves a class's private implementation details out of the header entirely, into a forward-declared, incomplete `Impl` class whose real definition lives only in the `.cpp` file. Two concrete payoffs: (1) **faster compilation at scale** — changing a private member no longer forces every file that includes `Widget.h` to recompile, since the header's own content didn't actually change; (2) **genuine encapsulation at the binary level** — callers can't even see the private implementation's types, not just be blocked from accessing them by `private` (Section 1.2) alone. The one non-obvious requirement: the destructor *must* be defined in the `.cpp` file, not defaulted inline in the header — `unique_ptr<Impl>`'s destructor needs `Impl`'s complete definition to know how to destroy it, which doesn't exist yet at the point the header is read.

```
struct EmptyDeleter { void operator()(int* p) const { delete p; } }; // no data members

struct NaiveHolder {
    int* ptr;
    EmptyDeleter deleter; // as a MEMBER: still takes >= 1 byte, even though it holds no data
};
struct OptimizedHolder : private EmptyDeleter { // as a BASE CLASS: can take 0 extra bytes (EBO)
    int* ptr;
};
```

An empty class (no data members) still normally requires at least 1 byte of storage as a *member*, so that two distinct objects don't end up at the same address. But the same rule doesn't have to apply to an empty **base class subobject** — the standard explicitly permits (and real compilers apply) the Empty Base Optimization: an empty base class can occupy zero bytes, since it never needs a distinct address separate from the derived object it's part of. This is exactly why standard-library tools like custom allocators or deleters (as in a `unique_ptr` with a stateless custom deleter) are commonly implemented via private inheritance rather than as a plain member — it can make the resulting type genuinely smaller.

CHAPTER 10 RECAP

- Base constructs first, derived destructs first — always
- A virtual call inside a constructor/destructor dispatches to *that class's own version*, never a more-derived override
- C++17 makes RVO mandatory for a returned prvalue; earlier standards only permitted it
- A `shared_ptr` cycle leaks silently — break it with one `weak_ptr` in the cycle
- CRTP gets you compile-time "polymorphism" with zero vtable cost, at the price of a uniform runtime collection
- PIMPL trades an extra indirection for faster builds and true binary-level encapsulation

11 Practice Problems

Small exercises first, to isolate one concept each; then three small design problems that combine several chapters at once, the way a real interview question would.

Warm-up exercises (isolate one concept each)

- **Exercise 1 (Ch. 2, 2.5):** Write a Base/Derived pair where deleting a Derived* through a Base* leaks memory. Then fix it with one keyword.
- **Exercise 2 (Ch. 3.4):** Write a function that takes a Shape by value and demonstrate, with printed output, that it slices a Circle passed into it.
- **Exercise 3 (Ch. 4.1–4.2):** Write a Rule-of-5 class wrapping a raw int*, then rewrite it as Rule-of-0 using std::unique_ptr<int> instead.
- **Exercise 4 (Ch. 5.1–5.2):** Reproduce the diamond-inheritance ambiguity, then fix it with virtual inheritance.
- **Exercise 5 (Ch. 6.2–6.4):** Write operator+, operator==, and operator<< for a small Vector2D {double x, y;} class.

Mini design 1: Logger

Requirements

Design a logging system with multiple output destinations (console, file) and multiple severity levels (INFO, WARNING, ERROR), where the destination is swappable at runtime.

Class design: An abstract LogSink interface (Section 1.6/2.4) with a pure virtual write(level, message); concrete ConsoleSink and FileSink implementations; a Logger class that *composes* (Section 5.3, not inherits) a std::unique_ptr<LogSink>, since a logger "has-a" destination rather than "is-a" one.

```
enum class Level { Info, Warning, Error };

class LogSink {
public:
    virtual void write(Level level, const std::string& msg) = 0;
    virtual ~LogSink() = default;    // Ch.2.5 -- required, this hierarchy is deleted polymorphically
};

class ConsoleSink : public LogSink {
public:
    void write(Level level, const std::string& msg) override {
        std::cout << msg << "\n";
    }
};

class Logger {
public:
    explicit Logger(std::unique_ptr<LogSink> sink) : sink_(std::move(sink)) {} // Rule of 0, Ch.4.1
    void log(Level level, const std::string& msg) { sink_->write(level, msg); }
    void setSink(std::unique_ptr<LogSink> sink) { sink_ = std::move(sink); } // swappable at runtime
private:
    std::unique_ptr<LogSink> sink_;
};
```

Extension — "now support logging to multiple sinks at once": change `sink_` from a single `unique_ptr<LogSink>` to a `std::vector<std::unique_ptr<LogSink>>`, and have `log()` loop over all of them — the interface (`LogSink`) needs no changes at all, only `Logger`'s internals, because callers only ever depended on the abstraction (Section 1.6).

Mini design 2: Shape hierarchy

Requirements

Design a set of 2D shapes (circle, rectangle, triangle) that each report their area and perimeter, printable in a uniform way, stored together in one collection.

Class design: Abstract `Shape` base with pure virtual `area()`, `perimeter()`, and a virtual `print(std::ostream&)`; a free operator<< (Section 6.4) that calls the virtual `print`, so every shape prints correctly through one uniform stream operator regardless of its concrete type.

```
class Shape {
public:
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual void print(std::ostream& os) const = 0;
    virtual ~Shape() = default;
};

std::ostream& operator<<(std::ostream& os, const Shape& s) { s.print(os); return os; }

class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159 * r_ * r_; }
    double perimeter() const override { return 2 * 3.14159 * r_; }
    void print(std::ostream& os) const override { os << "Circle(r=" << r_ << ")"; }
private:
    double r_;
};

std::vector<std::unique_ptr<Shape>> shapes; // Ch.3.4 -- pointers, to avoid slicing
shapes.push_back(std::make_unique<Circle>(2.0));
for (const auto& s : shapes) std::cout << *s << " area=" << s->area() << "\n";
```

Extension — "now add a Group shape that contains other shapes": a `CompositeShape : public Shape` holding a `std::vector<std::unique_ptr<Shape>>`, whose `area()` sums its children's areas — this is the Composite design pattern, and it works precisely because `CompositeShape` both *is* a `Shape` (inheritance) and *has* other `Shapes` (composition), combining Chapters 2 and 5 in one class.

Mini design 3: Notification system

Requirements

Design a small notification system supporting multiple channels (email, SMS) where sending a notification through any channel that fails should be retried, without duplicating the retry logic per channel.

Class design: An abstract Notifier interface with a pure virtual `send(message)`; concrete `EmailNotifier/SmsNotifier`; a `RetryingNotifier` that *wraps* any other Notifier (the Decorator idea — composition, not inheritance, giving it retry behavior without duplicating it per-channel).

```
class Notifier {
public:
    virtual bool send(const std::string& message) = 0;
    virtual ~Notifier() = default;
};

class RetryingNotifier : public Notifier {
public:
    RetryingNotifier(std::unique_ptr<Notifier> inner, int maxAttempts)
        : inner_(std::move(inner)), maxAttempts_(maxAttempts) {}
    bool send(const std::string& message) override {
        for (int attempt = 0; attempt < maxAttempts_; ++attempt)
            if (inner_->send(message)) return true;
        return false;
    }
private:
    std::unique_ptr<Notifier> inner_; // composes ANY Notifier -- works for email, SMS, or future channels
    int maxAttempts_;
};

// Usage: wrap any concrete notifier with retry behavior, no duplication
auto notifier = std::make_unique<RetryingNotifier>(
    std::make_unique<EmailNotifier>(), 3);
```

Extension — "now add rate limiting so we never send more than 10 messages/minute per channel": add another wrapper, `RateLimitedNotifier : public Notifier`, composed the same way — and stack it: `RateLimitedNotifier(RetryingNotifier(EmailNotifier()))`. Each concern (retrying, rate limiting) is an independent, composable wrapper around the base interface, rather than a growing pile of special cases inside one class.

- Compose behavior via wrapping (Decorator-style) instead of duplicating it per concrete subclass
- Store polymorphic collections as `vector<unique_ptr<Base>>`, never `vector<Base>`
- A class can simultaneously *be* its base type and *have* instances of it (Composite)

12 Quick Revision & Glossary

A last-pass-before-the-interview compression of the whole book.

THE FOUR PILLARS, ONE LINE EACH

- **Encapsulation** — bundle data + behavior, protect an invariant (Ch.1.5)
- **Abstraction** — expose intent, hide implementation, via an abstract class (Ch.1.6)
- **Inheritance** — "is-a," `public` by default, favor composition otherwise (Ch.2.1, 5.3)
- **Polymorphism** — same interface, different behavior; static (overload) or dynamic (virtual) (Ch.2.2)

THE INTERNALS, COMPRESSED

- One `vp`tr per polymorphic *object*; one `vtable` per polymorphic *class* (Ch.3.2)
- Virtual call = object → `vp`tr → `vtable` slot → indirect call
- No virtual destructor + delete via base pointer = derived destructor never runs (Ch.2.5)
- Pass-by-value into a base parameter slices a derived object (Ch.3.4)
- A virtual call inside a ctor/dtor uses *that class's own* version, never a more-derived one (Ch.10, Q17)

RULE OF 0/3/5

- Rule of 0 (preferred): write none of the five — delegate to members that already manage themselves
- Rule of 3/5: write all of them, together, the moment you manage a raw resource yourself
- `unique_ptr` default; `shared_ptr` for genuine shared ownership; `weak_ptr` to break cycles

Comparison tables, all in one place

COMPARE	LEFT	RIGHT
Overloading vs. overriding	Compile-time, same name/different params	Runtime, same signature, virtual
Static vs. dynamic binding	Decided at compile time	Decided at runtime via vtable
Deep vs. shallow copy	Duplicates pointed-to data	Copies the pointer value only
Composition vs. inheritance	"Has-a," loose coupling	"Is-a," tighter coupling
unique_ptr vs. shared_ptr	Exclusive, zero overhead	Shared, refcounted, can cycle-leak

Glossary

Class / Object

A type definition (blueprint) vs. one concrete instance of it.

Invariant

A condition that must always hold true for an object to be valid; the reason encapsulation exists.

Virtual function

A member function resolved at runtime via the vtable, based on the object's actual type.

Vtable / vptr

Per-class array of function pointers / per-object pointer into that array — the mechanism behind dynamic dispatch.

Object slicing

Losing a derived object's extra data and vtable when copied by value into a base-typed variable.

RAII

Resource Acquisition Is Initialization — tie a resource's lifetime to an object's constructor/destructor.

Move semantics

Transferring ownership of a resource from a temporary, instead of copying it.

Copy elision / RVO

The compiler constructing a value directly in its final location, skipping intermediate copies/moves — mandatory in C++17 for a returned prvalue.

CRTP

Curiously Recurring Template Pattern — compile-time "polymorphism" via a class inheriting from a template of itself.

PIMPL

Pointer to Implementation — hiding a class's private members behind a forward-declared pointer, for faster builds and true binary encapsulation.

Diamond problem

Ambiguity from multiple inheritance producing two copies of a shared ancestor; fixed with virtual inheritance.

```
[ ] Re-derive the vtable diagram from memory (Ch.3.2) without looking
[ ] Explain the virtual-destructor bug AND why it happens, not just that it's bad (Ch.2.5)
[ ] Trace the constructor/destructor virtual-call gotcha out loud (Ch.10, Q17)
[ ] State the Rule of 0/3/5 and WHY Rule of 0 is preferred (Ch.4.1)
[ ] Draw all four UML relationships from memory (Ch.9)
[ ] Write a shared_ptr cycle and its weak_ptr fix from scratch (Ch.10, Q14)
[ ] Practice one mini design (Ch.11) fully, end to end, in under 20 minutes
```

How to convert this guide to PDF

This guide is delivered two ways: as this print-ready page, and as a companion single-file LaTeX document.

- **This page (Markdown/HTML → PDF):** click the "Save as PDF" button at the top of the page, or use your browser's own Print (Ctrl/Cmd+P) and choose "Save as PDF" as the destination — the page's print styling is already tuned for clean pagination, wrapped code blocks, and no cut-off tables.
- **The LaTeX edition (.tex → PDF):** compile with `pdflatex` twice (the second pass resolves the table of contents and internal cross-references) — works directly on Overleaf with no setup, or locally with any full TeX Live / MiKTeX install.

The C++ OOP Handbook · Concepts, Internals & Interview Preparation · Targets C++17, with C++11/14/20 notes called out explicitly throughout.